

Inference Engineering Week Task

Part A – Theory

1. Difference Between Latency and Throughput in Model Inference

Latency and throughput are two important measures used to understand the performance of a machine learning model during inference. They are related, but they measure different aspects of performance.

Latency is the amount of time taken by the model to process one request and produce an output. It is usually measured in milliseconds (ms). For example, if a model takes 100 milliseconds to process a request, its inference latency is 100 ms.

Throughput refers to the number of requests or samples that a model can process in a given amount of time. It is usually measured in requests per second or samples per second. For example, if a model can process 50 samples in one second, its throughput is 50 samples per second.

The importance of latency and throughput depends on the type of application. For real-time applications such as chatbots or voice assistants, latency is usually more important because the user expects a quick response. On the other hand, for applications that process a large number of inputs, such as batch processing, throughput becomes more important.

Batch size can also affect model performance. Increasing the batch size allows multiple inputs to be processed together, which can improve throughput. However, processing a larger batch may take more total time. Therefore, there is usually a trade-off between latency and throughput.

2. KV Cache and How It Helps During Generation

KV cache stands for Key-Value cache. It is an optimization commonly used by Transformer-based language models when generating text one token at a time.

During the attention calculation, the model creates Key (K) and Value (V) representations for the tokens that have already been processed. When the next token is generated, the model still needs the information from those previous tokens.

Without a KV cache, the model would repeatedly calculate the Key and Value representations of the previous tokens. This results in unnecessary computation.

With a KV cache, the previously calculated Key and Value tensors are stored in memory and reused during the next generation step.

The process can be understood in the following way:

1. The model processes the input tokens.
2. Key and Value tensors are calculated for these tokens.
3. These Key and Value tensors are stored in the KV cache.
4. When another token needs to be generated, the stored values are reused.
5. Only the new token needs new Key and Value calculations.

For example, suppose the model has already processed the sentence:

“The cat is sleeping”

When generating the next token, the model can reuse the Key and Value information of the previous tokens instead of calculating them again.

Therefore, KV caching reduces repeated calculations and makes text generation faster. This becomes especially useful when the input or generated sequence is long.

The main drawback is that the cached Key and Value tensors occupy additional memory.

3. Quantization and Its Trade-off with Model Accuracy

Quantization is a technique used to represent model weights or activations using fewer bits than the original representation. It is mainly used to reduce model size and memory usage and can also help improve inference efficiency.

For example, a model may originally store its weights using FP32, where each value requires 32 bits. Quantization can convert these values to lower-precision formats such as INT8 or INT4.

INT8 Quantization	INT4 Quantization
Uses 8 bits to represent each value.	Uses 4 bits to represent each value.
Requires more memory than INT4.	Requires less memory than INT8.
Usually preserves model accuracy better.	Can introduce more quantization error and may reduce accuracy more.
Provides a good balance between memory usage and accuracy.	Provides greater memory savings but has a higher risk of accuracy loss.

Table 1: INT8 and INT4 quantization comparison

Quantization Formula

One simple way of performing symmetric quantization is to first calculate a scale value:

$$S = \frac{\max(|x|)}{127} \quad (1)$$

where x represents the original floating-point values.

The floating-point value can then be converted into an integer using:

$$q = \text{round} \left(\frac{x}{S} \right) \quad (2)$$

Example Calculation

Suppose the model contains the following weights:

$$x = [-1.0, -0.5, 0, 0.5, 1.0]$$

The largest absolute value is:

$$\max(|x|) = 1.0$$

Therefore, the scale for INT8 quantization is:

$$S = \frac{1.0}{127}$$

$$S \approx 0.007874$$

Now consider the value $x = 0.5$.

Using the quantization formula:

$$q = \text{round} \left(\frac{0.5}{0.007874} \right)$$

$$q \approx 64$$

To convert it back to an approximate floating-point value:

$$\hat{x} = 64 \times 0.007874$$

$$\hat{x} \approx 0.5039$$

So, the original value was 0.5, while the reconstructed value is approximately 0.5039. The small difference between these two values is the quantization error.

Another way to see the example is:

This shows why quantization can slightly change the values stored in a model. INT8 normally gives a reasonable balance between memory reduction and accuracy, while INT4 provides even greater memory savings but can cause more quantization error.

4. Inference Serving Framework

One real inference serving framework is **vLLM**. It is designed to make the serving of large language models more efficient.

One of the important optimizations used by vLLM is **PagedAttention**. It manages the KV cache in smaller blocks of memory rather than requiring a large continuous memory area for every sequence.

This improves memory utilization because memory can be allocated and managed more efficiently. It also allows more requests to be handled at the same time, which can improve the throughput of an LLM serving system.